3. Modern AI Araç Setimiz: Copilot, Cursor, Claude Code ve Ötesi

Hangi Görev İçin Hangi AI Aracını Seçmelisin?
Bölüm 1 • Zorluk Derecesi: Başlangıç Seviyesi • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Cursor varken Copilot gereksiz mi? Ya da Claude Code terminal asistanları geliştiricinin yerini mi alıyor?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Her yapay zeka aracının bir 'tatlı noktası' (sweet spot) vardır; tek bir araca bağımlı kalmayın.
> - 📌 Satır içi tamamlama (Ghost Text) için Copilot, tüm kod tabanı indeksleme için Cursor, otonom refactoring için CLI agent'lar idealdir. - 📌 En pahalı araç değil, bağlamı (context) en iyi yönettiğiniz araç en iyi sonucu verir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Marangoz atölyesine girdiğinizi düşünün. Duvarda testere, rende, zımpara ve çekiç asılı. Bir marangoz vida sıkmak için çekiç kullanırsa ne olur? Tahta çatlar ve vida bozulur.

Yazılım dünyasında da bugün benzer bir hata yapılıyor: Bir junior, 50 dosyalık bir mimari refactoring'i GitHub Copilot'un satır içi otomatik tamamlamasına yaptırmaya çalışıyor. Veya basit bir `console.log` eklemek için 5 dakika otonom CLI agent çalıştırıp jeton (token) yakıyor.

Doğru aleti doğru işte kullanmak, profesyonel bir mühendisi amatörden ayıran ilk beceridir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Araçlarının Katmanlı Mimarisi:

Katman	Araç Örnekleri	En İyi Kullanım Alanı	Dikkat Edilmesi Gereken
Micro (Satır İçi)	GitHub Copilot, Supermaven	Boilerplate tamamlama, döngüler, DTO tanımları	Körü körüne Tab'a basıp yanlış mantığı onaylamak
Meso (Editör İçi)	Cursor (Composer), Windsurf	Dosyalar arası ilişki, `@files`, `@docs` bağlamı	Çok fazla gereksiz dosya ekleyip token limitini aşmak
Macro (CLI / Agent)	Claude Code, Aider, Antigravity	Terminal komutları, testleri çalıştırıp otomatik düzeltme	Yanlış komut çalıştırma veya git geçmişini bozma riski

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Senior Full-Stack Mimar
Projemizde PostgreSQL, Prisma ORM, Next.js ve TailwindCSS kullanıyoruz.

## BAĞLAM (@schema.prisma ve @auth.ts):
Aşağıdaki yeni gereksinimi karşılamak için hangi dosyaları değiştirmem gerektiğini
listele:
"Kullanıcıların profil sayfasına iki faktörlü kimlik doğrulama (2FA) açma/kapama
butonu ekleme."

Lütfen hemen kod yazma; sadece:
1. Değişecek veritabanı alanları
2. Backend API endpoint'i
3. Frontend bileşeni
adım adım bir mimari plan sun.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Yaklaşım: AI'ın çalıştırdığı terminal komutunu kontrol etmeden çalıştırmak
# Yapay zekaya "Projeyi sıfırla ve yeniden kur" dediniz ve şunu önerdi:
rm -rf node_modules package-lock.json && git clean -fdx && npm install
# DİKKAT: Henüz commit etmediğiniz tüm yerel dosyalarınız sonsuza dek silindi!
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Güvenli Yaklaşım: Önce durumu doğrula, dry-run yap ve korumalı çalış
# 1. Değişiklikleri güvene al
git status
git stash save "proje-temizlik-oncesi"

# 2. Sadece hedeflenen paketleri temizle
rm -rf node_modules
npm cache verify
npm install

# 3. Testleri çalıştırarak ortamı doğrula
npm test
```

💡 **Kod Analizi:** Terminal üzerinde çalışan otonom AI agent'ları kullanırken `git clean -fdx` veya `rm -rf` gibi yıkıcı komutları asla gözünüz kapalı onaylamayın. Her zaman önce `git stash` veya branch yedeği alın.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Aynı anda 3 farklı AI eklentisini açıp birbirleriyle çakıştırmak.
  - **Çözüm:** Birincil editörünüzü ve tek bir tamamlama aracını belirleyip odaklanın.
- **Tuzak:** Büyük projelerde tüm kod tabanını prompta yapıştırmak.
  - **Çözüm:** Sadece ilgili servis ve interface dosyalarını bağlam (@file) olarak ekleyin.
- **Tuzak:** AI CLI araçlarına sudo veya sınırsız terminal izni vermek.
  - **Çözüm:** Hassas terminal komutları için her zaman manuel onay modunu açık tutun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: *Ders 3: AI Araç Kutunuzu Kurmak - Copilot, Cursor ve CLI Entegrasyonu*

- 🖥️ **Slayt 1:** 3 Katmanlı AI Mimarisi: Micro, Meso, Macro araçlar.
- 🖥️ **Slayt 2:** Cursor'da @sembolleri (@files, @docs, @codebase) ile bağlam yönetimi.
- 🖥️ **Slayt 3:** Terminal asistanlarını güvenli kullanma kılavuzu.
- 🖥️ **Slayt 4:** Canlı karşılaştırma: Hangi araç nerede kazandırır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Cursor IDE'de `@codebase` kullanarak projedeki bir veritabanı modelinin nerede kullanıldığını sorgulatın. Ardından `@file` ile sadece iki dosyayı seçerek hedefli bir refactor gerçekleştirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün IDE'nizde en sık kullandığınız 3 dosyayı `@` kısayoluyla bağlama ekleyerek küçük bir fonksiyon geliştirin ve token kullanımını gözlemleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Cursor veya benzeri modern AI IDE'lerinde '@' sembolünün temel amacı nedir? - A) E-posta göndermek - B) Modele yalnızca belirli dosyaları, dokümanları veya kod tabanını bağlam olarak iletmek - C) Kodu otomatik olarak derlemek - D) GitHub şifresini kaydetmek > **Doğru Yanıt:** B > **Açıklama:** @ sembolü sayesinde LLM'e projenin sadece ilgili parçaları bağlam olarak gönderilir; böylece hem maliyet düşer hem de doğruluk artar.

Soru 2: Otonom terminal agent'ları (CLI) kullanırken en kritik güvenlik kuralı nedir? - A) İnternet bağlantısını kesmek - B) Dosya silen ve git geçmişini etkileyen komutları manuel olarak denetlemek - C) Sadece Python kullanmak - D) Bilgisayarı yeniden başlatmak > **Doğru Yanıt:** B > **Açıklama:** Terminal agent'ları geri dönülemez dosya silme veya branch ezme komutları önerebilir. Kritik komutlar mutlaka insan onayından geçmelidir.